

Hybrid Python Reading Plan

AI / ML Track

Revised edition v2 — tightened ordering, absorbed thin phase, added checkpoint projects and a testing/debugging module.

26 core chapters from **Fluent Python (2nd ed.)** and **High Performance Python (2nd ed.)**, ordered for maximum leverage in AI/ML work.

26	6	~17 wk	4	11
core chapters	phases	estimated	checkpoint projects	skipped / skimmed

FP

Fluent Python

HPP

High Performance Python

■ *tightening note*

HOW TO USE THIS PLAN

1. Read sequentially within each phase. Phases build on each other; do not jump ahead.
2. Within a phase, follow the listed order. Pairings are intentional — 'what' before 'how fast'.
3. Code every chapter. Passive reading yields ~20% of the value. Open a notebook and break things.
4. Hit every checkpoint project. They are the difference between 'read a book' and 'can ship'.
5. Skipped chapters are reference, not garbage. Return to them when a real problem demands it.

WHAT CHANGED FROM v1

Phase 2: FP Ch 6 (mutability) moved up — tensor aliasing bugs poison everything downstream.

Phase 3: FP Ch 8 (types) now precedes FP Ch 9 (decorators) — modern decorator signatures are typed.

Phase 6: Eliminated as standalone. HPP Ch 7 (compiling) absorbed into Phase 4 where it pairs naturally with vectorization.

Phase 7: Regrouped — parallelism block (FP 19 → FP 20 → HPP 9) then async block (FP 21 → HPP 8). No more context-switching.

FP Ch 4: Demoted from SKIP to SKIM. Tokenization for LLMs is byte-level work.

New module: Testing & debugging mini-phase between core content and concurrency (off-book, but essential).

New: Four explicit checkpoint projects with concrete deliverables.

THE 6 PHASES

PHASE 01 The Foundation

~2 weeks · 3 chapters

Goal: Understand how Python and the machine actually work before writing a single optimization.

FP

Ch 1

The Python Data Model

Dunder methods — the grammar underlying every Python framework you'll use.

HPP

Ch 1

Understanding Performant Python

How CPUs, memory units, and the Python VM relate — the mental model for everything else.

HPP

Ch 2

Profiling to Find Bottlenecks

Measure first, always. cProfile, line_profiler, memory_profiler — drives every optimization decision.

PHASE 02 Core Data Structures

~3 weeks · 6 chapters

Goal: Know what your ML data actually lives in, and the performance cost of each choice.

FP

Ch 2

An Array of Sequences

Lists, tuples, arrays — pick the right container for your data pipeline.

FP

Ch 3

Dictionaries and Sets

The backbone of model configs, vocab maps, feature stores, label encoders.

FP

Ch 6

Object References & Mutability

Stop silent bugs when sharing tensors between functions — copies vs views.

■ *Moved up from end of phase. Mutability misunderstandings poison everything downstream.*

HPP

Ch 3

Lists and Tuples (performance)

Memory layout — why tuples are faster for fixed data, how CPython allocates.

HPP

Ch 4

Dictionaries and Sets (performance)

Hash tables internals, O(1) lookups, memory cost — understand what your dicts are doing.

FP

Ch 5

Data Class Builders

Clean dataclasses for experiment configs, dataset records, hyperparameter bundles.

★ CHECKPOINT PROJECT — after Phase 2: Build a Dataset and DataLoader from scratch

Using only the standard library, implement a class that loads CSV records, supports indexing and length, and yields shuffled minibatches via a generator. No NumPy, no PyTorch. Deliverable: working module + 5 unit tests + a README explaining why you chose list vs tuple vs dict for each internal structure.

PHASE 03 Functions, Types & Decorators

~2 weeks · 3 chapters

Goal: Understand the core pattern every ML framework (PyTorch, JAX, TF) is built on.

FP Ch 7 Functions as First-Class Objects

Higher-order functions, closures — the foundation of functional ML APIs.

FP Ch 8 Type Hints in Functions

Production ML code needs type safety — enables mypy, better IDE support, safer refactors.

■ *Now BEFORE decorators. Modern decorator signatures use typed protocols.*

FP Ch 9 Decorators and Closures

@torch.no_grad, @tf.function, @lru_cache — now you fully understand why they work.

PHASE 04 Iterators, Numerical Computing & Compilation

~4 weeks · 5 chapters

Goal: Build memory-efficient data pipelines, master NumPy/Pandas, and escape Python's speed ceiling when vectorization isn't enough.

FP Ch 17 Iterators, Generators & Classic Coroutines

Lazy evaluation for large datasets — the architectural basis of PyTorch DataLoader.

HPP Ch 5 Iterators and Generators (performance)

Generators vs lists: memory profiling on real ML data volumes — when laziness saves you.

HPP Ch 6 Matrix and Vector Computation

NumPy vectorization, numexpr, Pandas performance — the beating heart of all ML code.

HPP Ch 11 Using Less RAM

Train on bigger datasets without OOM — array.array, np.float32, object reuse strategies.

HPP Ch 7 Compiling to C

Cython, Numba, Ctypes — write custom ops, speed up preprocessing, understand PyTorch extensions.

■ *Absorbed from the eliminated Phase 6. Pairs naturally with vectorization: when vectorizing isn't enough, compile.*

★ CHECKPOINT PROJECT — after Phase 4: NumPy-only minibatch trainer + Numba speedup

Write a linear regression trainer using only NumPy. Profile it with cProfile and line_profiler. Identify the slowest pure-Python loop. Beat it twice: once with vectorization, once with @numba.jit. Deliverable: notebook with three timings (naive / vectorized / compiled) and a paragraph explaining the speedup ratio of each.

Goal: Build proper ML components: models, datasets, trainers, and evaluation loops.

FP Ch 11 **A Pythonic Object**
`__repr__`, `__len__`, `__eq__` — the public interface every ML component should implement.

FP Ch 12 **Special Methods for Sequences**
 Build your own Dataset class with `__getitem__` and `__len__` — you finally understand why.

FP Ch 13 **Interfaces, Protocols & ABCs**
 Duck typing vs strict contracts — how PyTorch `nn.Module` and sklearn estimators are designed.

FP Ch 18 **with, match, and else Blocks**
 Context managers for GPU memory, file handles, experiment tracking — use `torch.cuda.amp` cleanly.

★ **CHECKPOINT PROJECT — after Phase 5: Tiny nn.Module-style API**

Implement a minimal neural-network base class with `__call__`, `parameters()`, `state_dict()`, `load_state_dict()`, and `__repr__`. Build a 2-layer MLP on top of it and train on a toy regression task with manual backprop (or autograd library of your choice). Deliverable: ~150 LOC framework + working training script.

Goal: The biggest gap between 'read Fluent Python' and 'ship ML systems'. Neither book covers this; do not skip.

Topic 1 **pytest fundamentals** — fixtures, parametrize, monkeypatch. Write 10 tests against your Phase 5 checkpoint code.

Topic 2 **pdb / ipdb** — breakpoints, post-mortem, navigating frames. Drop into a deliberately buggy training loop and find the issue.

Topic 3 **PyTorch-specific debugging** — `torch.autograd.set_detect_anomaly`, gradient checking, NaN traps, shape mismatch diagnostics.

Topic 4 **Reproducibility** — seeding, deterministic flags, snapshotting environments with `pip-compile` or `uv`.

Goal: Parallelize data loading, training pipelines, and ML model serving APIs.

Block A — Parallelism (processes & threads)

FP

Ch 19 **Concurrency Models in Python**

GIIL, threads, processes — the full truth behind PyTorch DataLoader's num_workers.

FP

Ch 20 **Concurrent Executors**

ThreadPoolExecutor, ProcessPoolExecutor — batch inference, parallel preprocessing.

HPP

Ch 9 **The multiprocessing Module**

Shared memory, worker pools — tune num_workers properly, avoid pickling traps.

Block B — Async I/O

FP

Ch 21 **Asynchronous Programming**

async/await — serve ML models via FastAPI or async endpoints without blocking.

HPP

Ch 8 **Asynchronous I/O**

Event loops, async file reads — maximize disk throughput during training data loading.

■ *Moved here from earlier position. Now flows directly from async/await fundamentals.*

★ CHECKPOINT PROJECT — after Phase 6: FastAPI inference endpoint with warm process pool

Wrap a trained model behind a FastAPI async endpoint. Use a ProcessPoolExecutor with persistent workers to handle CPU-bound inference. Load-test with httpx or locust. Deliverable: working API + load test results showing throughput vs naive synchronous baseline + a paragraph on why processes (not threads) for inference.

CHAPTERS TO SKIP OR SKIM

*Note: FP Ch 4 was a hard skip in v1 — promoted to **SKIM** here. If you'll touch LLMs at all, tokenization is byte-level work.*

FP

Ch 4 — Unicode Text vs Bytes

SKIM — read the bytes/strings split + encoding gotchas if doing any NLP or LLM tokenization work.

FP

Ch 10 — Design Patterns

Academic value, low practical ML return — bookmark for later.

FP

Ch 14 — Inheritance Pitfalls

Good theory, rarely the bottleneck in ML codebases.

FP

Ch 15 — More About Type Hints

Skim when you hit generics in a real project.

FP

Ch 16 — Operator Overloading

Mostly useful for custom numeric types — reference only.

FP

Ch 22 — Dynamic Attributes & Properties

Optional — relevant when building frameworks, not using them.

FP**Ch 23 — Attribute Descriptors**

Deep internals — skip unless writing metaclass-heavy code.

FP**Ch 24 — Class Metaprogramming**

Fascinating, but far from typical ML workflows.

HPP**Ch 10 — Clusters & Job Queues**

Read when you move to distributed training (Dask, Ray, Celery).

HPP**Ch 12 — Lessons from the Field**

Good stories — save for a long flight.

Generated for AI/ML practitioners — v2 revision. **Fluent Python 2nd ed.** (Ramalho, O'Reilly 2022) · **High Performance Python 2nd ed.** (Gorelick & Ozsvald, O'Reilly 2020).